Table des matières

Exercices corrigés 06 — Agrégation avancée et pivots 1
Questions courtes 1
Questions pratiques 3
Diagnostics 4
Questions à développer 5
Mises en situation 5
Corrigé 7

Exercices corrigés 06 — Agrégation avancée et pivots

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez quatre heures et trois quarts pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué `[OLTP]` se traite dans PostgreSQL sur `Adventureworks` (schémas `sales`, `production`, `person`). Un énoncé marqué `[OLAP]` se traite dans DuckDB sur `adventureworksdw.duckdb`, ouvert par `USE dbo;`. Sans marque, la question vaut pour les deux.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Le manque du `GROUP BY`. Qu'est-ce qu'un `GROUP BY` ordinaire ne sait pas produire, et que les extensions de ce cours produisent en une seule passe ?
2. La solution naïve. Avant `ROLLUP`, comment obtenait-on le détail et ses sous-totaux ? Donnez deux défauts de cette méthode.
3. `GROUPING SETS`. Que désigne une paire de parenthèses vides `()` dans la liste, et pourquoi la notation est-elle cohérente ?
4. `ROLLUP`. Écrivez le `GROUPING SETS` équivalent à `ROLLUP (a, b, c)`, dans l'ordre.
5. L'ordre compte. Qu'est-ce qui change entre `ROLLUP (annee, pays)` et `ROLLUP (pays, annee)` ?
6. `CUBE`. Combien de niveaux produit `CUBE` sur n colonnes, et quelle conséquence pratique en tirez-vous avant d'y mettre quatre colonnes ?

© 2026 iAlexMG. Tous droits réservés.

7. Choisir. Vous voulez le détail par catégorie et sous-catégorie, plus le sous-total par catégorie, plus le total. Lequel des trois mots-clés prenez-vous, et pourquoi pas les deux autres ?

8. Le `NULL` ambigu. Dans un résultat de `ROLLUP`, qu'est-ce qui distingue une ligne où `pays` vaut `NULL` parce que c'est un sous-total d'une ligne où le pays est réellement inconnu ?

9. `GROUPING()`. Que renvoie cette fonction, et sur quel argument s'applique-t-elle ?

10. Le tri des totaux. Pourquoi trie-t-on souvent par `GROUPING(col)` avant de trier par la colonne elle-même ?

11. L'agrégat conditionnel. Expliquez le mécanisme de `SUM(CASE WHEN cond THEN mesure END)` en une phrase, en disant précisément ce que devient une ligne qui ne remplit pas la condition.

12. Le `ELSE` absent. Pourquoi n'écrit-on pas `ELSE 0` dans ce `CASE` ? Dans quel cas le choix change-t-il le résultat ?

13. Compter sous condition. Écrivez l'expression qui compte les lignes remplissant une condition, et dites pourquoi `COUNT(CASE WHEN cond THEN 1 END)` ne se remplace pas par `COUNT(*)`.

14. `[OLTP]` La clause `FILTER`. Écrivez l'équivalent de la question 11 avec `FILTER`, et donnez son avantage.

15. La portabilité de `FILTER`. `FILTER` passe-t-il sur les deux moteurs de cette formation ? Est-ce pour autant une écriture qu'on peut employer sans y penser ? Dites ce qui rend cette dette particulièrement discrète.

16. `PIVOT`. Nommez les trois choses qu'il faut fournir à un `PIVOT` sous sa forme standard, et dites laquelle ne peut pas venir des données. L'entrepôt offre une seconde forme qui lève cette contrainte : dites ce qu'elle coûte en échange.

1. Le piège de la sous-requête source. Pourquoi ne met-on que les colonnes utiles dans la sous-requête d'un `PIVOT`, et que se passe-t-il si on en laisse une de trop ?

2. `UNPIVOT`. Que fait-il, à quoi sert le format qu'il produit, et quel piège guette celui qui replie un tableau large reçu d'ailleurs ?

3. `[OLTP]` L'absence de `PIVOT`. PostgreSQL n'a pas `PIVOT`. Donnez la raison technique, puis les trois voies possibles.

4. Le format long. Qu'est-ce que c'est, et quel argument décisif joue en sa faveur quand la liste des colonnes peut changer ?

5. `[OLTP]` Le mot réservé. Quelle colonne d'AdventureWorks OLTP oblige à écrire des guillemets doubles dans ce cours, et pourquoi ?

6. `[OLTP]` L'alias dans le `ROLLUP`. PostgreSQL accepte `GROUP BY ROLLUP (annee, pays)` quand `annee` est un alias défini dans le `SELECT`. Pourquoi ne pas compter sur cette tolérance, et quelle construction du Cours 05 rend la question sans objet ?

7. Le pays en clair. Sur l'entrepôt, le nom du pays est une colonne de la dimension ; sur le transactionnel, c'est un code à joindre. Qu'est-ce que cet écart dit du travail d'alimentation ?

© 2026 iAlexMG. Tous droits réservés.

8. Ce qui est standard. Parmi `ROLLUP`, `CUBE`, `GROUPING SETS`, `GROUPING()`, `FILTER`, `PIVOT`, `DISTINCT ON` et `QUALIFY`, dites lesquels sont présents des deux côtés, et rangez les autres par moteur.
9. `STRING_AGG`. Que rend-il, en quoi répond-il à une autre question que `COUNT`, et pourquoi son `ORDER BY` s'écrit-il à l'intérieur de l'agrégat ?
10. La liste qui bégaie. Une liste de couleurs agrégée à travers une table de faits contient des milliers d'éléments pour cinq couleurs. D'où viennent-ils, et quel mot les élimine ?

Questions pratiques

Écrivez la requête complète avant de regarder. Quand un énoncé existe en deux versions, faites les deux et comparez-les : c'est là que le contraste entre les deux mondes devient concret.

P1. `[OLAP]` La cascade continent puis pays. Avec `ROLLUP`, affichez les ventes par continent (`SalesTerritoryGroup`) puis par pays, avec les sous-totaux par continent et le total général.

P2. `[OLTP]` La même cascade, l'autre voie. Reproduisez P1 sur le transactionnel. Vous aurez besoin de deux jointures de plus et des guillemets du labo.

P3. `[OLAP]` Les quantités par continent, en colonnes. Comme en 2.3 du labo, mais affichez la quantité vendue (`SUM(OrderQuantity)`) par année, une colonne par continent.

P4. `[OLTP]` Les quantités par continent, avec `FILTER`. La même chose, écrite avec la clause `FILTER` plutôt qu'avec des `CASE`.

P5. `[OLAP]` Le pivot par catégorie de produit. Faites pivoter les catégories (`EnglishProductCategoryName` : `'Bikes'`, `'Components'`, `'Clothing'`, `'Accessories'`) en colonnes, les années en lignes. Vous devrez joindre `DimProduct`, `DimProductSubcategory` et `DimProductCategory`.

P6. `[OLTP]` Le même tableau, sans `PIVOT`. Reproduisez P5 sur le transactionnel, avec l'écriture que vous jugez la meilleure. Justifiez votre choix en une phrase.

P7. Le total étiqueté. Reprenez P1 et remplacez les `NULL` de sous-total par « Tous continents » et « Tous pays » à l'aide de `GROUPING()`. Donnez la version de votre terrain préféré.

P8. La part dans le sous-total. Pour chaque pays, affichez ses ventes et sa part dans les ventes de son continent. Deux approches sont possibles, l'une avec `ROLLUP`, l'autre avec une fonction de fenêtrage du Cours 04 : donnez celle que vous préférez et dites pourquoi.

P9. `[OLAP]` Le tableau croisé à deux mesures. Affichez, par année, le montant et la quantité pour chacun des trois continents, soit six colonnes de mesure. Dites ce que `PIVOT` vous impose ici.

P10. `(avancé)` Le comptage sous conditions multiples. Pour chaque année, affichez le nombre de commandes distinctes, le nombre de celles à plus de 1 000 dollars, et la proportion des secondes dans les premières. Une seule passe sur les données.

P11. `(avancé)` Le grain qui dérape. Écrivez volontairement le `PIVOT` de P5 avec une colonne de trop dans la sous-requête source, constatez le résultat, puis expliquez précisément le mécanisme qui produit ces lignes supplémentaires. Donnez ensuite la règle que vous en tirez.

© 2026 iAlexMG. Tous droits réservés.

P12. (avancé) Le pivot dynamique. On vous demande un tableau croisé dont les colonnes sont les sous-catégories de produits, sans en écrire aucune en dur, sachant que la liste peut changer. Écrivez-le sur l'entrepôt, qui sait le faire, et comptez les colonnes obtenues : le compte devrait vous surprendre, dites pourquoi. Expliquez ensuite pourquoi PostgreSQL refuse par principe de faire la même chose, et ce que son refus protège. Terminez en proposant deux solutions défendables pour un rapport qui doit durer, et dites laquelle vous mettriez en production.

P13. [OLAP] Les couleurs des vélos. Pour chaque sous-catégorie de vélos (`ProductCategoryKey` vaut 1), affichez les couleurs vendues en ligne, une seule fois chacune et dans l'ordre alphabétique, en un seul texte.

P14. [OLTP] Combien de raisons. Comptez les commandes selon leur nombre de raisons d'achat, une, deux ou trois, à partir de `sales.salesorderheadersalesreason`.

Diagnostics

On vous montre du code qui ne fait pas ce que son auteur croit. Dites ce qui cloche, pourquoi, et donnez le correctif.

E1. Un analyste veut le détail par année et par pays, plus le sous-total par année. Il écrit :

```sql
SELECT d.CalendarYear AS annee, t.SalesTerritoryCountry AS pays,
       SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY CUBE (d.CalendarYear, t.SalesTerritoryCountry)
ORDER BY annee, pays;
```

Le résultat contient bien ce qu'il demandait, et il le livre en fichier. Sa collègue l'ouvre dans un tableur, somme la colonne `ventes` et trouve 117 434 708,88 $, quand l'entrepôt en compte 29 358 677,22 $ en tout. Dites ce que la requête produit en trop et donnez le correctif ; puis appliquez votre correctif, refaites la somme de la collègue, et dites si le désaccord est réglé. Terminez par les deux façons de le régler vraiment.

E2. Une développeuse veut le montant total et le montant des seules ventes européennes :

```sql
SELECT d.CalendarYear AS annee,
       SUM(f.SalesAmount) AS total,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Europe' THEN f.SalesAmount ELSE 0 END) AS europe,
       AVG(CASE WHEN t.SalesTerritoryGroup = 'Europe' THEN f.SalesAmount ELSE 0 END) AS moy_europe
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY d.CalendarYear;
```

La colonne `europe` est juste, la colonne `moy_europe` est absurdement basse. Expliquez pourquoi la même écriture donne un résultat juste dans un cas et faux dans l'autre, et corrigez.

© 2026 iAlexMG. Tous droits réservés.

E3. Un rapport doit distinguer les lignes de sous-total :

```
SELECT
    COALESCE(t.SalesTerritoryCountry, 'Total') AS pays,
    SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY ROLLUP (t.SalesTerritoryCountry);
```

L'auteur trouve son écriture plus courte que le `CASE WHEN GROUPING(...)` du cours, et il a raison sur la longueur. Dites dans quelles conditions précises sa requête devient fausse, pourquoi cette faute est particulièrement difficile à repérer, et donnez le correctif.

E4. `[OLTP]` Un contrôle de nuit compare la liste des raisons d'achat de chaque commande à celle de la veille, et signale toute différence :

```
SELECT hr.salesorderid, STRING_AGG(r.name, ', ') AS raisons
FROM sales.salesorderheadersalesreason AS hr
JOIN sales.salesreason                 AS r ON r.salesreasonid = hr.salesreasonid
GROUP BY hr.salesorderid;
```

Certaines nuits, il signale des commandes dont aucune raison n'a changé. Dites ce qui cloche et corrigez.

Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.

A. Le rapport et le grain. Un `ROLLUP` produit, dans un même résultat, des lignes qui ne sont pas au même grain : du détail et des agrégats mélangés. Expliquez pourquoi c'est commode pour un rapport et dangereux pour tout ce qui consomme ce résultat en aval. Donnez au moins deux façons dont un traitement automatique peut se tromper là-dessus, et dites ce que vous feriez pour l'en protéger.

B. Le pivot, en SQL ou ailleurs. On peut croiser des données dans la requête, ou livrer du format long et laisser l'outil d'affichage croiser. Défendez d'abord le pivot en SQL, puis l'argument inverse, et concluez par la règle que vous adopteriez dans une équipe qui alimente à la fois des tableaux de bord et des exports de fichiers.

C. La portabilité, jusqu'où. Ce cours contient à la fois le morceau le plus portable de la formation (les extensions du `GROUP BY`) et l'un des moins portables (`PIVOT`). Expliquez comment vous décidez, en pratique, d'utiliser ou non une facilité propre à un moteur. Distinguez le cas d'une équipe mono-moteur de celui d'une équipe qui migre, et dites ce que vous écririez dans une convention d'équipe.

Mises en situation

On décrit une entreprise et une question métier. À vous de choisir l'approche, de l'écrire, et surtout de la justifier. Il y a plusieurs bonnes réponses, mais il y a des réponses indéfendables.

© 2026 iAlexMG. Tous droits réservés.

M1. Réseau de quincailleries Beauport, le rapport du lundi matin. Le réseau compte vingt-deux succursales réparties en quatre régions, et vend six familles de produits. La direction veut un seul rapport hebdomadaire qui montre, pour chaque région, le chiffre de chaque succursale, le sous-total de la région, le total du réseau, et à droite une colonne par famille de produits. Le rapport doit tenir sur une page et se lire sans formation.

Décrivez la requête complète : où va le `ROLLUP`, où vont les colonnes de famille, et dans quel ordre les deux se combinent. Le point difficile est de mêler un pivot et des lignes de sous-total : dites pourquoi ces deux besoins se gênent, et comment vous les faites cohabiter. Terminez sur l'étiquetage : que mettez-vous dans la colonne « succursale » sur la ligne de sous-total d'une région, et pourquoi ce détail n'est pas cosmétique.

M2. Coopérative agricole Saint-Hyacinthe, le tableau qui grandit. La coopérative suit les livraisons de ses membres par culture. Un rapport existant croise les cultures en colonnes, écrites en dur dans un `PIVOT` : blé, maïs, soya, avoine. Cette année, trois membres se sont mis au sarrasin. Le rapport continue de tourner sans erreur, et personne ne s'aperçoit de rien pendant deux mois.

Expliquez exactement pourquoi aucune erreur n'a été levée, et ce que le rapport a montré pendant ces deux mois. Puis proposez une reconception. Vous devez traiter trois questions : comment détecter automatiquement qu'une valeur nouvelle est apparue, faut-il ou non que le rapport s'adapte tout seul, et où placer la responsabilité du croisement. Prenez position, et dites ce que vous répondez au gestionnaire qui demande simplement « ajoutez le sarrasin ».

M3. `(avancé)` Groupe hôtelier Charlevoix, l'audit du chiffre. Le groupe exploite neuf établissements. Le service financier produit un rapport mensuel avec sous-totaux par établissement, par région et pour le groupe, à partir d'un entrepôt. Le service des opérations produit son propre tableau à partir de la base de réservation. Les deux totaux généraux diffèrent de trois pour cent, et chaque service accuse l'autre.

Décrivez la démarche que vous suivriez pour trancher, en SQL. Il ne s'agit pas d'écrire une requête mais une méthode : comment on compare deux agrégats produits par deux chaînes différentes, à quel grain on descend, et dans quel ordre. Traitez explicitement les quatre causes classiques d'un écart de ce genre, dont au moins une tient aux sujets de ce cours. Terminez en disant ce que vous recommanderiez pour que cette situation ne se reproduise pas, sachant que supprimer un des deux rapports n'est pas une option acceptable pour la direction.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés.

Questions courtes

1. Un `GROUP BY` ordinaire ne produit qu'un seul niveau de regroupement. Les extensions produisent plusieurs niveaux à la fois, donc le détail, ses sous-totaux et le total général dans un même résultat, en une seule passe sur les données.

Théorie : `Theorie_06_SQL.md`, section 1.

2. On écrivait une requête par niveau et on les collait avec `UNION ALL`. Deux défauts. C'est long, et surtout les requêtes se désynchronisent : le jour où l'on ajoute un filtre, il faut penser à le répéter partout, et un oubli donne un total qui ne correspond plus à son détail. Second défaut, on balaie les données autant de fois qu'il y a de niveaux, alors qu'une seule passe suffirait.

Théorie : `Theorie_06_SQL.md`, section 1.

3. Les parenthèses vides désignent le total général. La notation est cohérente parce que chaque parenthèse liste les colonnes qui définissent un niveau : ne rien lister, c'est ne regrouper sur rien, donc regrouper tout le jeu de données en une seule ligne.

Théorie : `Theorie_06_SQL.md`, section 2.

4. `GROUPING SETS ((a, b, c), (a, b), (a), ())`.

Théorie : `Theorie_06_SQL.md`, sections 2 et 3.

5. La direction de la cascade. `ROLLUP (annee, pays)` donne le détail par année et pays, puis un sous-total par année, puis le total. `ROLLUP (pays, annee)` donne le même détail, mais un sous-total par pays. La liste des lignes de détail est identique, celle des lignes de synthèse ne l'est pas.

Théorie : `Theorie_06_SQL.md`, section 3.

6. Deux puissance n niveaux. Sur quatre colonnes, cela fait seize niveaux, donc un résultat qui mélange seize grains différents et qui devient à la fois coûteux et illisible. La conséquence pratique est qu'on ne met dans un `CUBE` que les axes qu'on veut vraiment croiser, et qu'au-delà de deux ou trois, il vaut mieux lister à la main avec `GROUPING SETS`.

Théorie : `Theorie_06_SQL.md`, section 4.

7. `ROLLUP (categorie, sous_categorie)`, parce que les deux colonnes forment une hiérarchie naturelle et que c'est exactement la cascade demandée. `GROUPING SETS` donnerait le même résultat mais en trois lignes d'écriture au lieu d'une, sans rien apporter. `CUBE` produirait en plus le sous-total par sous-catégorie toutes catégories confondues, ce qui n'a aucun sens ici, puisqu'une sous-catégorie n'appartient qu'à une catégorie.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_SQL.md`, section 2.

8. Rien, dans le résultat brut. Les deux lignes portent un `NULL` indiscernable, et c'est précisément le problème que la fonction `GROUPING()` résout.

Théorie : `Theorie_06_SQL.md`, section 5.

9. Elle renvoie 1 si la colonne passée en argument est agrégée sur cette ligne, donc si la ligne est un sous-total pour cet axe, et 0 sinon. Elle s'applique à une colonne du `GROUP BY`, pas à une expression quelconque.

Théorie : `Theorie_06_SQL.md`, section 5.

10. Attention au piège de la question : tant que les sous-totaux restent des `NULL` bruts, un `ORDER BY annee, pays` suffit sur nos deux terrains, qui rangent l'un comme l'autre les `NULL` en fin de tri croissant, donc chaque sous-total juste après son bloc. Mesuré des deux côtés. Le tri par `GROUPING(col)` devient nécessaire à la ligne d'après, celle où l'on remplace les `NULL` par des libellés : « Tous pays » n'est plus un `NULL` mais du texte, et le tri le range par son initiale, entre « Germany » et « United Kingdom », au beau milieu du bloc qu'il résume. Trier d'abord sur `GROUPING(col)`, qui vaut 0 pour le détail et 1 pour la synthèse, remet chaque sous-total au bas de son bloc quel que soit son libellé. La règle à retenir n'est donc pas « les `NULL` se rangent mal », mais « dès qu'on étiquette une ligne de synthèse, on ne peut plus la trier par ce qu'elle affiche ».

Théorie : `Theorie_06_SQL.md`, section 5.

11. La fonction `CASE` produit la mesure pour les lignes qui remplissent la condition et `NULL` pour les autres, et `SUM` ignore les `NULL`. Une ligne qui ne remplit pas la condition n'est donc pas écartée du regroupement, elle y participe avec une valeur nulle qui ne pèse rien.

Théorie : `Theorie_06_SQL.md`, section 6.

12. Parce que `ELSE 0` change la nature de ce qui est ignoré. Sur un `SUM`, on dit souvent que la faute est sans effet, puisque ajouter des zéros ne change pas un total : c'est vrai du montant, et faux du tableau. Dans un groupe où aucune ligne ne remplit la condition, la version sans `ELSE` rend une case vide et la version avec `ELSE 0` rend un zéro, ce qui n'est pas la même information : « ce pays n'a rien vendu cette année-là » n'est pas « il a vendu, pour zéro dollar ». Sur un `AVG`, un `COUNT` ou un `MIN`, la faute est franche : les zéros deviennent des valeurs comptées, la moyenne est tirée vers le bas par des lignes qui n'avaient rien à faire dans le calcul, et le compte devient celui du groupe entier. Sur l'entrepôt, les ventes allemandes de 2011 se comptent à 175 sans `ELSE` et à 2 216 avec. La règle est donc d'omettre le `ELSE` toujours, et pas seulement quand on sait que ça compte ; le zéro d'affichage, si on le veut, se pose à l'extérieur par un `COALESCE`.

Théorie : `Theorie_06_SQL.md`, section 6.

13. `COUNT(CASE WHEN cond THEN 1 END)`. Le remplacement par `COUNT(*)` est impossible parce que `COUNT(*)` compte les lignes du groupe sans regarder aucune valeur, y compris celles qui ne remplissent pas la condition. C'est `COUNT(expression)` qui ignore les `NULL`, et c'est cette propriété que le motif exploite.

Théorie : `Theorie_06_SQL.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

14. `[OLTP]` `SUM(mesure) FILTER (WHERE cond)`. L'avantage est la lisibilité : la clause dit littéralement ce qu'elle fait, alors que le `CASE` oblige le lecteur à reconstituer le mécanisme des `NULL` ignorés. Le gain devient net dès qu'il y a plusieurs colonnes conditionnelles côte à côte.

Théorie : `Theorie_06_SQL.md`, section 6.

15. Oui, `FILTER` passe sur les deux moteurs de la formation, et il fait partie de la norme SQL : la même mesure rend 8 930 042 $ pour l'Europe, écrite en `FILTER` comme en `SUM(CASE …)`.

Ce n'est pas pour autant une écriture qu'on emploie sans y penser. La norme ne garantit pas la présence, et `FILTER` manque encore à plusieurs moteurs répandus, dont Snowflake d'après sa documentation. La dette est ici particulièrement discrète, et c'est le point de la question : elle ne se manifeste sur aucun des deux terrains que vous pratiquez. Une équipe qui teste des deux côtés et voit tout passer en conclut naturellement que l'écriture est sûre, et découvre le contraire le jour de la migration. On s'en tient donc au `SUM(CASE …)` dans tout ce qui doit voyager, et on réserve `FILTER` au code qui reste chez soi.

Théorie : `Theorie_06_SQL.md`, sections 6 et 9.

16. La mesure et sa fonction d'agrégat, la colonne à éclater en colonnes, et la liste des valeurs attendues. C'est la troisième qui ne peut pas venir des données : elle doit être écrite en dur, parce que le moteur a besoin de connaître la forme du résultat avant de l'exécuter.

Théorie : `Theorie_06_SQL.md`, section 7.

17. Parce que toute colonne présente dans la sous-requête et absente du `PIVOT` sert de clé de regroupement implicite. Si on en laisse une de trop, le résultat éclate en une ligne par valeur de cette colonne : le tableau reste correct au sens du calcul, mais il n'est plus au grain demandé. Aucune erreur n'est levée, ce qui en fait un piège sérieux.

Théorie : `Theorie_06_SQL.md`, section 7.

18. `UNPIVOT` replie des colonnes en lignes : un tableau large avec une colonne par année devient deux colonnes, l'année et la valeur. Le format produit est le format long, celui qui se regroupe, se filtre et se joint facilement, donc celui qu'on veut en entrée de tout traitement.

Son piège tient en une phrase : une case vide ne donne pas de ligne. Le repli du tableau croisé du labo, cinq années sur quatre pays, rend dix-neuf lignes et non vingt, parce que l'Allemagne n'a aucune vente en 2010. Rien ne le signale. C'est sans conséquence quand on replie pour ré-agréger, puisqu'une somme ignore de toute façon les valeurs nulles ; c'en a une dès qu'on compte les couples, qu'on joint, ou qu'on vérifie qu'un tableau reçu est complet. La parade sur l'entrepôt est un `COALESCE` par colonne repliée, qui oblige au passage à décider ce que le trou voulait dire.

Théorie : `Theorie_06_SQL.md`, section 7.

19. `[OLTP]` La raison technique est que PostgreSQL exige de connaître le type et le nombre de colonnes du résultat avant l'exécution, alors qu'un pivot dont les colonnes viendraient des données ne les connaît qu'après. Les trois voies sont l'agrégat conditionnel, la fonction `crosstab` de l'extension `tablefunc`, et le renoncement au pivot en SQL au profit du format long croisé par l'outil d'affichage.

© 2026 iAlexMG. Tous droits réservés.

Une mise en garde sur la deuxième, que le labo fait mesurer : `crosstab` à un argument remplit les colonnes dans l'ordre où les lignes arrivent, sans regarder le nom de la catégorie. Une catégorie absente d'un groupe décale toutes celles qui la suivent, et le rapport attribue alors des chiffres justes aux mauvaises colonnes, sans la moindre erreur. On ne s'en sert qu'à deux arguments, en fournissant la liste des catégories attendues.

Théorie : `Theorie_06_SQL.md`, sections 7 et 9.

20. Le format long est une ligne par combinaison d'axes, avec les valeurs empilées dans une seule colonne de mesure au lieu d'être étalées en colonnes. L'argument décisif est qu'il ne se casse pas : une valeur nouvelle apparaît comme une ligne de plus, alors qu'un tableau large aux colonnes écrites en dur l'ignorera sans rien signaler.

Théorie : `Theorie_06_SQL.md`, section 7.

21. `[OLTP]` La colonne `group` de `sales.salesterritory`, qui porte le continent. `GROUP` est un mot réservé du langage, donc le moteur l'interprète comme le début d'un `GROUP BY` s'il n'est pas protégé. On écrit `t."group"`, avec des guillemets doubles, qui délimitent un identifiant et non une chaîne de caractères.

Théorie : `Theorie_06_SQL.md`, section 9.

22. `[OLTP]` Parce que c'est une tolérance de PostgreSQL, pas une règle du langage : un moteur qui suit la norme à la lettre refuse l'alias comme la position dans un `GROUP BY`, l'un et l'autre naissant après lui. Même sur PostgreSQL, un alias qui reprend le nom d'une colonne des tables lues désigne la colonne, et non l'expression. La CTE du Cours 05 rend la question sans objet : à l'intérieur de la requête qui la consomme, `annee` n'est plus un alias mais une véritable colonne du résultat intermédiaire.

Théorie : `Theorie_06_SQL.md`, sections 3 et 9. Théorie : `Theorie_02_SQL.md`, section 7.

23. Que l'alimentation d'un entrepôt fait un travail de résolution qu'on ne voit plus une fois qu'il est fait. Quelqu'un a joint le code au référentiel des pays, choisi le libellé de référence, et gelé le résultat dans la dimension. Chaque analyste économise cette jointure, et surtout tous les analystes obtiennent le même libellé, ce qui est l'apport le moins visible et le plus important.

Théorie : `Theorie_06_SQL.md`, section 9.

24. Présents des deux côtés : `ROLLUP`, `CUBE`, `GROUPING SETS`, `GROUPING()`, et aussi `FILTER` et `DISTINCT ON`, qu'on croit souvent propres à PostgreSQL. Propres à l'entrepôt dans le cadre de cette formation : `PIVOT`, `UNPIVOT` et `QUALIFY`.

La liste des écarts se réduit donc à trois mots, et c'est le bilan à retenir de ce cours : tout ce qui relève du `GROUP BY` voyage, seul le pivot ne voyage pas.

Théorie : `Theorie_06_SQL.md`, section 9.

25. Il rend un texte, les valeurs du groupe recollées avec un séparateur : il dit « lesquelles » là où `COUNT` dit « combien ». L'`ORDER BY` se place à l'intérieur parce qu'il trie les valeurs à l'intérieur de chaque groupe, pas les lignes du résultat ; sans lui, la norme ne garantit aucun ordre, et deux listes faites des mêmes valeurs peuvent ne pas être égales.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_SQL.md`, section 8.

26. La jointure avec les faits répète chaque couleur une fois par ligne de vente : 15 205 éléments pour les cinq couleurs des vélos vendus en ligne. `DISTINCT`, écrit dans l'agrégat, `STRING_AGG(DISTINCT p.Color, ', ' ORDER BY p.Color)`, ne garde chaque valeur qu'une fois.

Théorie : `Theorie_06_SQL.md`, section 8.

Questions pratiques

P1. [OLAP]

```sql
SELECT t.SalesTerritoryGroup AS continent, t.SalesTerritoryCountry AS pays,
       SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY ROLLUP (t.SalesTerritoryGroup, t.SalesTerritoryCountry)
ORDER BY continent, pays;
```

Théorie : `Theorie_06_SQL.md`, section 3.

P2. [OLTP]

```sql
WITH ventes AS (
    SELECT t."group" AS continent, c.name AS pays, d.linetotal AS montant
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    JOIN sales.salesterritory   AS t ON t.territoryid  = h.territoryid
    JOIN person.countryregion   AS c ON c.countryregioncode = t.countryregioncode
)
SELECT continent, pays, SUM(montant) AS ventes
FROM ventes
GROUP BY ROLLUP (continent, pays)
ORDER BY continent, pays;
```

Quatre tables au lieu de deux, et un mot réservé à protéger. Le `ROLLUP`, lui, ne bouge pas d'un caractère.

Théorie : `Theorie_06_SQL.md`, sections 3 et 9.

P3. [OLAP]

```sql
SELECT d.CalendarYear AS annee,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'North America' THEN f.OrderQuantity END) AS qte_amerique_nord,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Europe'        THEN f.OrderQuantity END) AS qte_europe,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Pacific'       THEN f.OrderQuantity END) AS qte_pacifique
```


© 2026 iAlexMG. Tous droits réservés.

```sql
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY d.CalendarYear
ORDER BY annee;
```

Théorie : `Theorie_06_SQL.md`, section 6.

P4. [OLTP]

```sql
WITH ventes AS (
    SELECT EXTRACT(YEAR FROM h.orderdate)::int AS annee,
           t."group" AS continent, d.orderqty AS quantite
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    JOIN sales.salesterritory   AS t ON t.territoryid  = h.territoryid
)
SELECT annee,
       SUM(quantite) FILTER (WHERE continent = 'North America') AS qte_amerique_nord,
       SUM(quantite) FILTER (WHERE continent = 'Europe')        AS qte_europe,
       SUM(quantite) FILTER (WHERE continent = 'Pacific')       AS qte_pacifique
FROM ventes
GROUP BY annee
ORDER BY annee;
```

Théorie : `Theorie_06_SQL.md`, section 6.

P5. [OLAP]

```sql
SELECT *
FROM (
    SELECT d.CalendarYear AS annee, c.EnglishProductCategoryName AS categorie,
           f.SalesAmount AS montant
    FROM dbo.FactInternetSales     AS f
    JOIN dbo.DimDate               AS d ON d.DateKey = f.OrderDateKey
    JOIN dbo.DimProduct            AS p ON p.ProductKey = f.ProductKey
    JOIN dbo.DimProductSubcategory AS s ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
    JOIN dbo.DimProductCategory    AS c ON c.ProductCategoryKey = s.ProductCategoryKey
) AS src
PIVOT (SUM(montant) FOR categorie IN ('Bikes', 'Components', 'Clothing', 'Accessories')) AS p
ORDER BY annee;
```

Théorie : `Theorie_06_SQL.md`, section 7.

© 2026 iAlexMG. Tous droits réservés.

P6. [OLTP]

```sql
WITH ventes AS (
    SELECT EXTRACT(YEAR FROM h.orderdate)::int AS annee,
           pc.name AS categorie, d.linetotal AS montant
    FROM sales.salesorderheader          AS h
    JOIN sales.salesorderdetail          AS d  ON d.salesorderid = h.salesorderid
    JOIN production.product              AS p  ON p.productid = d.productid
    JOIN production.productsubcategory   AS ps ON ps.productsubcategoryid = p.productsubcategoryid
    JOIN production.productcategory      AS pc ON pc.productcategoryid = ps.productcategoryid
)
SELECT annee,
       SUM(montant) FILTER (WHERE categorie = 'Bikes')       AS velos,
       SUM(montant) FILTER (WHERE categorie = 'Components')  AS composants,
       SUM(montant) FILTER (WHERE categorie = 'Clothing')    AS vetements,
       SUM(montant) FILTER (WHERE categorie = 'Accessories') AS accessoires
FROM ventes
GROUP BY annee
ORDER BY annee;
```

Je choisis `FILTER` parce que la requête reste sur nos moteurs, qui l'acceptent tous les deux, et que la lisibilité prime ; si la même requête devait un jour partir vers un moteur qui l'ignore, j'écrirais `SUM(CASE …)` dès maintenant, pour ne pas avoir à y revenir.

Théorie : `Theorie_06_SQL.md`, sections 7 et 9.

P7. Version entrepôt.

```sql
SELECT
    CASE WHEN GROUPING(t.SalesTerritoryGroup) = 1 THEN 'Tous continents'
         ELSE t.SalesTerritoryGroup END AS continent,
    CASE WHEN GROUPING(t.SalesTerritoryCountry) = 1 THEN 'Tous pays'
         ELSE t.SalesTerritoryCountry END AS pays,
    SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY ROLLUP (t.SalesTerritoryGroup, t.SalesTerritoryCountry)
ORDER BY GROUPING(t.SalesTerritoryGroup), t.SalesTerritoryGroup,
         GROUPING(t.SalesTerritoryCountry), t.SalesTerritoryCountry;
```

Théorie : `Theorie_06_SQL.md`, section 5.

P8. Je préfère la fenêtre, et de loin.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT t.SalesTerritoryGroup AS continent, t.SalesTerritoryCountry AS pays,
       SUM(f.SalesAmount) AS ventes,
       SUM(f.SalesAmount) / SUM(SUM(f.SalesAmount)) OVER (PARTITION BY t.SalesTerritoryGroup) AS part
FROM dbo.FactInternetSales AS f
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY t.SalesTerritoryGroup, t.SalesTerritoryCountry
ORDER BY continent, part DESC;
```

La raison du choix tient au grain. La version `ROLLUP` obligerait à produire les lignes de sous-total, puis à les rejoindre au détail pour obtenir un dénominateur, donc à mélanger deux grains dans une même requête avant de les séparer de nouveau. La fenêtre garde une seule ligne par pays et calcule le total du continent à côté, ce qui est exactement le geste du Cours 04. C'est aussi le rappel que `ROLLUP` sert à afficher des totaux, pas à s'en servir dans un calcul.

Notez l'agrégat imbriqué, `SUM(SUM(...)) OVER (...)` : le `SUM` intérieur agrège au grain du `GROUP BY`, l'extérieur est une fenêtre posée sur ce résultat. C'était la question 21 du Cours 04.

Théorie : `Theorie_06_SQL.md`, sections 1 et 3.

P9. `[OLAP]` Sous sa forme standard, celle du cours, `PIVOT` ne prend qu'une seule fonction d'agrégat. Pour deux mesures, il faut donc deux `PIVOT` et une jointure, ou bien renoncer et écrire six agrégats conditionnels. La seconde voie est la plus simple à relire, et c'est celle qui suit.

L'entrepôt a pourtant de quoi faire autrement, et il vaut la peine de le voir avant de choisir. Sa forme propre accepte plusieurs agrégats d'un coup :

```sql
PIVOT (
    SELECT d.CalendarYear AS annee, t.SalesTerritoryGroup AS continent,
           f.SalesAmount AS montant, f.OrderQuantity AS qte
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
) ON continent USING SUM(montant), SUM(qte)
ORDER BY annee;
```

Elle rend bien les six colonnes de mesure demandées, en une passe. Le prix est dans leurs noms, que le moteur fabrique : `Europe_sum(montant)`, `North America_sum(qte)`, avec une parenthèse et un espace au milieu, donc des identifiants à mettre entre guillemets doubles partout en aval. Pour un tableau qu'on regarde, c'est parfait ; pour un rapport dont d'autres requêtes consomment les colonnes, les six agrégats conditionnels nommés à la main restent préférables. Ce que `PIVOT` impose ici n'est donc pas la même chose selon la forme : la standard impose de renoncer, celle du moteur impose de vivre avec ses noms de colonnes.

```sql
SELECT d.CalendarYear AS annee,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'North America' THEN f.SalesAmount END)   AS montant_an,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'North America' THEN f.OrderQuantity END) AS qte_an,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Europe'        THEN f.SalesAmount END)   AS montant_eu,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Europe'        THEN f.OrderQuantity END) AS qte_eu,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Pacific'       THEN f.SalesAmount END)   AS montant_pa,
```


© 2026 iAlexMG. Tous droits réservés.

```
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Pacific'       THEN f.OrderQuantity END) AS qte_pa
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY d.CalendarYear
ORDER BY annee;
```

La leçon est que la syntaxe dédiée gagne sur un cas simple et perd dès que le besoin se complique. L'agrégat conditionnel, lui, ne se dégrade jamais.

Théorie : `Theorie_06_SQL.md`, section 7.

P10. `(avancé)`

```
WITH commandes AS (
    SELECT d.CalendarYear AS annee, f.SalesOrderNumber,
           SUM(f.SalesAmount) AS montant_commande
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, f.SalesOrderNumber
)
SELECT annee,
       COUNT(*) AS nb_commandes,
       COUNT(CASE WHEN montant_commande > 1000 THEN 1 END) AS nb_grosses,
       COUNT(CASE WHEN montant_commande > 1000 THEN 1 END)::float / COUNT(*) AS part_grosses
FROM commandes
GROUP BY annee
ORDER BY annee;
```

Le point à ne pas manquer est le niveau. Le seuil de 1 000 dollars porte sur la commande entière, pas sur une ligne de commande, donc il faut d'abord totaliser par commande. Appliquer le `CASE` directement sur les lignes de la table de faits compterait des lignes, pas des commandes, et donnerait un résultat plausible et faux.

Théorie : `Theorie_06_SQL.md`, section 6.

P11. `(avancé)` En ajoutant `f.ProductKey` à la sous-requête source du `PIVOT` de P5, le résultat passe de quelques lignes à plusieurs centaines.

Le mécanisme est le suivant. Un `PIVOT` regroupe implicitement sur toutes les colonnes de la source qui ne sont ni la colonne pivotée ni la mesure. Il n'y a pas de `GROUP BY` écrit, mais il y en a un, et sa liste est déduite. Ajouter une colonne, c'est donc ajouter un axe de regroupement dont on n'avait pas conscience : le tableau reste juste ligne par ligne, mais il n'est plus au grain demandé, et les totaux d'une même année sont éparpillés sur plusieurs lignes.

La règle qui en découle : la source d'un `PIVOT` s'écrit toujours comme une projection explicite de trois choses, l'axe des lignes, l'axe à pivoter et la mesure. Jamais un `SELECT *`, jamais une table telle quelle. Et quand la source est une CTE partagée avec d'autres requêtes, on la reprojette avant le `PIVOT` plutôt que de la brancher directement.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_SQL.md`, section 7.

P12. `(avancé)` L'entrepôt le fait, et en une requête :

```
PIVOT (
    SELECT d.CalendarYear AS annee, s.EnglishProductSubcategoryName AS sous_cat,
           f.SalesAmount AS montant
    FROM dbo.FactInternetSales      AS f
    JOIN dbo.DimDate                AS d ON d.DateKey = f.OrderDateKey
    JOIN dbo.DimProduct             AS p ON p.ProductKey = f.ProductKey
    JOIN dbo.DimProductSubcategory  AS s ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
) ON sous_cat USING SUM(montant);
```

Le résultat rend 5 lignes et 18 colonnes, l'année plus dix-sept sous-catégories. C'est le compte qui devait surprendre : la base en compte trente-sept. Les vingt autres n'apparaissent pas parce qu'elles n'ont aucune vente en ligne, et le tableau ne le signale pas. Un pivot qui découvre ses colonnes ne montre pas ce qui existe, il montre ce qui a des données, ce qui n'est pas la même question et ne se remarque pas à la lecture.

PostgreSQL refuse la construction elle-même, et pas par manque de moyens. Le type d'un résultat SQL, c'est-à-dire le nombre et le nom de ses colonnes, y doit être connu à l'analyse de la requête, avant que la moindre donnée soit lue. Ce que son refus protège, c'est le contrat : tout ce qui consomme le résultat, une vue, un export, un contrôle de qualité, un rapport, sait à quoi s'attendre sans avoir exécuté quoi que ce soit. Un moteur qui accepte le pivot dynamique déplace ce risque sur vous plutôt que de le supprimer, et c'est exactement ce que les dix-sept colonnes au lieu de trente-sept viennent de montrer.

Reste donc la question posée : que met-on en production pour un rapport qui doit durer. Deux solutions défendables. La première est la génération de code : une première requête ramène la liste des sous-catégories, un script ou une macro construit le texte SQL du pivot, et ce texte est exécuté. C'est ce que font dbt et la plupart des outils de transformation, et c'est honnête, parce que la liste des colonnes est figée au moment de la construction et connue de tout ce qui suit. La seconde est de livrer le format long et de laisser l'outil d'affichage croiser.

Je mettrais en production la seconde. La génération de code fonctionne, mais elle crée un artefact dont la forme change sans prévenir : le jour où une sous-catégorie apparaît, tout ce qui consomme le résultat en aval, un rapport, un export, un contrôle de qualité, se retrouve devant une colonne qu'il ne connaît pas. Le format long, lui, encaisse la nouveauté par une ligne de plus, ce que n'importe quel consommateur sait traiter. Je réserverais la génération de code au cas où un format de fichier large est contractuel, par exemple un export attendu par un partenaire.

Théorie : `Theorie_06_SQL.md`, section 7.

P13. `[OLAP]`

```
SELECT s.EnglishProductSubcategoryName                    AS sous_categorie,
       STRING_AGG(DISTINCT p.Color, ', ' ORDER BY p.Color) AS couleurs
FROM dbo.FactInternetSales     AS f
JOIN dbo.DimProduct            AS p ON p.ProductKey = f.ProductKey
JOIN dbo.DimProductSubcategory AS s ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
WHERE s.ProductCategoryKey = 1
```


© 2026 iAlexMG. Tous droits réservés.

```
GROUP BY s.EnglishProductSubcategoryName
ORDER BY sous_categorie;
```

Trois lignes : Mountain Bikes (Black, Silver), Road Bikes (Black, Red, Yellow), Touring Bikes (Blue, Yellow). Le départ par `FactInternetSales` est ce qui fait de la liste celle des couleurs vendues ; parti de `DimProduct`, on aurait eu les couleurs du catalogue, y compris celles qu'aucune vente en ligne ne porte.

Théorie : `Theorie_06_SQL.md`, section 8.

P14. `[OLTP]`

```
SELECT nb_raisons, COUNT(*) AS commandes
FROM (SELECT salesorderid, COUNT(*) AS nb_raisons
      FROM sales.salesorderheadersalesreason
      GROUP BY salesorderid) AS par_commande
GROUP BY nb_raisons
ORDER BY nb_raisons;
```

18 530 commandes ont une raison, 4 329 en ont deux, 153 en ont trois, soit 23 012 commandes qui en ont au moins une. Le premier `GROUP BY` ramène la table de liaison au grain de la commande, le second compte les commandes par nombre de raisons : deux niveaux d'agrégation, donc deux requêtes emboîtées. `CARDINALITY(ARRAY_AGG(salesreasonid))` donnerait le même compte que `COUNT(*)`, par un détour.

Théorie : `Theorie_06_SQL.md`, section 8.

Diagnostics

E1. Sa requête utilise `CUBE` là où `ROLLUP` suffisait. Le `CUBE` produit, en plus des niveaux demandés, un sous-total par pays toutes années confondues : 41 lignes au lieu de 35. Ces lignes supplémentaires sont parfaitement correctes prises une à une, et c'est ce qui rend l'erreur sournoise, rien ne signale qu'elles sont là. Le correctif tient en un mot :

```
GROUP BY ROLLUP (d.CalendarYear, t.SalesTerritoryCountry)
```

Appliquez-le, et refaites la somme de la collègue : elle trouve maintenant 88 076 031,66 $, soit exactement trois fois le total de l'entrepôt au lieu de quatre. Le désaccord n'est pas réglé, il est réduit, et c'est toute la leçon du diagnostic. Le vrai défaut n'est pas le choix de `CUBE`, qui n'est qu'une maladresse : c'est qu'on somme une colonne qui mélange des grains. Chaque vente y figure au détail, dans son sous-total d'année et dans le total général, et aucun choix d'extension du `GROUP BY` n'y changera rien, puisque c'est exactement ce qu'on leur demande de faire.

Deux façons de le régler vraiment, selon ce que la collègue veut. Si elle veut un total, elle ne doit pas sommer ce fichier : la ligne du total général le lui donne déjà, et c'est à cela qu'elle sert. Si elle veut ré-agréger elle-même, il faut lui livrer de quoi trier les lignes, en ajoutant `GROUPING(d.CalendarYear) AS g_annee` et `GROUPING(t.SalesTerritoryCountry) AS g_pays` au `SELECT` : un filtre sur `g_annee = 0 AND g_pays = 0` lui rend les 29 lignes de détail, qui se somment juste. Retenez la règle dans ce sens : un résultat de `ROLLUP` ou de `CUBE` se lit, il ne s'additionne pas.

Théorie : `Theorie_06_SQL.md`, sections 3 et 4.

© 2026 iAlexMG. Tous droits réservés.

E2. La différence tient à ce que chaque fonction fait des zéros ajoutés par le `ELSE 0`.

`SUM` additionne : ajouter des zéros ne change pas un total, donc la colonne `europe` est juste par accident. `AVG` divise par le nombre de valeurs non nulles : les zéros deviennent des valeurs comptées, donc le dénominateur enfle de toutes les ventes non européennes, et la moyenne s'effondre. Ce n'est pas une moyenne des ventes européennes, c'est la somme des ventes européennes divisée par le nombre total de ventes.

La correction consiste à retirer le `ELSE 0`, dans les deux colonnes et pas seulement dans celle qui est fausse.

```sql
SELECT d.CalendarYear AS annee,
       SUM(f.SalesAmount) AS total,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Europe' THEN f.SalesAmount END) AS europe,
       AVG(CASE WHEN t.SalesTerritoryGroup = 'Europe' THEN f.SalesAmount END) AS moy_europe
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY d.CalendarYear;
```

C'est pour cette raison qu'on omet le `ELSE` par principe, même quand la fonction du moment est un `SUM` : la colonne d'à côté, écrite six mois plus tard par quelqu'un d'autre, sera peut-être un `AVG`, et l'habitude aura mis un zéro qui ne se voit pas.

Théorie : `Theorie_06_SQL.md`, section 6.

E3. Sa requête devient fausse dès qu'un pays est réellement `NULL` dans les données, ce qui arrive plus souvent qu'on ne le croit : une vente sans territoire renseigné, une ligne de reprise, un enregistrement importé sans référentiel. Le `COALESCE` étiquette alors cette ligne de détail « Total », exactement comme la ligne de synthèse. Le résultat contient deux lignes intitulées « Total » qui ne veulent pas dire la même chose, et rien ne permet de les distinguer.

La faute est difficile à repérer pour trois raisons cumulées. La requête est correcte tant que les données sont propres, donc elle passe la relecture et les premiers mois d'exploitation. Quand la faute se manifeste, elle prend la forme d'une ligne en double, ce qui ressemble à un problème de tri ou d'affichage plutôt qu'à un problème de logique. Et la ligne parasite porte un petit montant, donc elle ne saute pas aux yeux dans un total.

Le même défaut existe, plus rare mais réel, si une valeur des données vaut littéralement la chaîne « Total ».

```sql
SELECT
    CASE WHEN GROUPING(t.SalesTerritoryCountry) = 1 THEN 'Total'
         ELSE COALESCE(t.SalesTerritoryCountry, 'Pays inconnu') END AS pays,
    SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY ROLLUP (t.SalesTerritoryCountry);
```

Le correctif garde un `COALESCE`, mais à sa vraie place : `GROUPING()` répond à la question « cette ligne est-elle un sous-total », et le `COALESCE` à la question « cette donnée est-elle manquante ». Ce sont deux questions différentes, et il faut deux outils.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_SQL.md`, section 5.

E4. `[OLTP]` Le `STRING_AGG` n'a pas d'`ORDER BY`. L'ordre des raisons dans la liste dépend alors du chemin que le moteur prend pour lire les lignes, qui peut changer d'une nuit à l'autre après une mise à jour des statistiques ou un nouvel index. « Price, On Promotion » et « On Promotion, Price » sont la même commande, mais pas le même texte, et la comparaison les déclare différents. Le correctif trie dans l'agrégat :

```
SELECT hr.salesorderid, STRING_AGG(r.name, ', ' ORDER BY r.name) AS raisons
FROM sales.salesorderheadersalesreason AS hr
JOIN sales.salesreason                 AS r ON r.salesreasonid = hr.salesreasonid
GROUP BY hr.salesorderid;
```

Une liste qui sert à comparer doit être triée sur une clé qui ne se répète pas dans le groupe : ici le nom de la raison, unique par commande.

Théorie : `Theorie_06_SQL.md`, section 8.

Questions à développer

A. Un résultat de `ROLLUP` mélange des grains, et c'est précisément ce qu'on lui demande : un rapport veut voir le détail et ses totaux au même endroit, parce qu'un lecteur humain lit une page, pas une requête. La commodité est réelle, elle évite trois requêtes et une mise en page manuelle.

Le danger apparaît dès que le résultat cesse d'être lu par un humain. Tout ce qui consomme des lignes en supposant qu'elles sont homogènes se trompe. Un tableur qui somme une colonne compte chaque montant autant de fois qu'il y a de niveaux au-dessus de lui. Un outil de visualisation qui agrège pour dessiner un graphique produit des barres deux ou trois fois trop hautes, sans erreur ni avertissement. Un export chargé dans une autre base y injecte des lignes de synthèse qui deviendront indiscernables du détail au bout d'une semaine. Et un contrôle de qualité qui compte les lignes voit un nombre qui ne correspond à rien.

La protection tient en trois gestes. On ajoute au résultat les colonnes `GROUPING()`, ce qui donne à l'aval le moyen mécanique de filtrer le détail, et on les nomme explicitement plutôt que de les enterrer dans un `CASE`. On ne laisse jamais un résultat de `ROLLUP` sortir d'une couche de présentation : la table qui sert de source à d'autres traitements reste au grain fin, et les sous-totaux se calculent en bout de chaîne. Et quand un rapport doit vraiment être livré avec ses totaux, on le documente comme tel, avec une colonne qui dit le niveau de chaque ligne en toutes lettres.

Théorie : `Theorie_06_SQL.md`, sections 1 et 5.

B. Pour le pivot en SQL : il livre un résultat prêt à afficher, sans dépendre de ce que fait l'outil en aval. Quand le même tableau part vers un rapport, un courriel automatique et un fichier, croiser une seule fois à la source garantit que les trois destinations montrent la même chose. La syntaxe dédiée est aussi plus lisible qu'une pile d'agrégats conditionnels, et la logique reste versionnée avec le reste du code plutôt que cachée dans la configuration d'un outil.

L'argument inverse est plus fort qu'il n'en a l'air. Un pivot écrit en SQL fige la liste des colonnes, donc il fige une hypothèse sur les données, et cette hypothèse vieillit sans prévenir. Il rend aussi le résultat difficile à réutiliser : un tableau large ne se filtre pas, ne se joint pas commodément, et se ré-agrège mal. Enfin, il place une décision de présentation dans la couche de données, ce qui est précisément le mélange des responsabilités qu'une architecture propre cherche à éviter.

© 2026 iAlexMG. Tous droits réservés.

La règle que j'adopterais dans une équipe qui fait les deux : les tables et les vues livrées par la couche de données sont toujours au format long, sans exception. Le croisement se fait dans l'outil d'affichage, qui sait le faire et le refait à chaque rafraîchissement. La seule dérogation est l'export de fichier dont la structure large est imposée par un tiers, et dans ce cas le pivot vit dans un objet dédié, nommé pour ce qu'il est, avec un contrôle qui échoue si une valeur inattendue apparaît. La dérogation s'écrit dans la convention, sinon elle devient la règle en six mois.

Théorie : `Theorie_06_SQL.md`, section 7.

C. La décision se prend sur une seule question : ce code a-t-il une chance réelle de devoir tourner ailleurs. Pas « est-ce que ce serait bien », mais « est-ce probable ». Une facilité de moteur n'est pas une faute, c'est une dette, et une dette se contracte en connaissance de cause.

Dans une équipe mono-moteur qui n'a aucun projet de migration, refuser `QUALIFY` ou `FILTER` est du purisme coûteux : on paie tous les jours en lisibilité pour un événement qui n'arrivera peut-être jamais. On les utilise, et on note quelque part qu'on les utilise.

Ce cours ajoute une nuance que le Cours 04 ne donnait pas. Certaines facilités passent sur les deux moteurs de la formation sans être pour autant portables, `FILTER` en tête : la dette existe, elle est simplement invisible depuis chez vous. Tester des deux côtés ne remplace donc pas la question de départ, et c'est le genre de faux sentiment de sécurité qu'il faut nommer avant qu'il ne décide à votre place.

Dans une équipe qui migre ou qui vit sur deux moteurs, le calcul s'inverse, mais pas pour tout le code. La ligne de partage utile n'est pas entre requêtes portables et non portables, elle est entre les couches. Ce qui est partagé, ce qui sera relu et repris, ce qui définit un modèle réutilisé par d'autres, doit rester dans le sous-ensemble commun aux deux moteurs. Ce qui est terminal, une requête ponctuelle, un rapport propre à une plateforme, un script d'exploration, peut utiliser tout ce que le moteur offre.

Dans une convention d'équipe, j'écrirais trois choses. La liste explicite des constructions réservées à un moteur, tenue à jour, parce qu'une règle qui commence par « tout ce qui n'est pas standard » ne se vérifie pas. La couche où chacune est autorisée, en une phrase. Et le geste à faire quand on veut déroger, à savoir un commentaire dans le code qui dit pourquoi, ce qui permet au moins de retrouver toutes les dérogations le jour de la migration.

Théorie : `Theorie_06_SQL.md`, section 9.

Mises en situation

M1. Réseau de quincailleries Beauport.

La requête se construit en deux couches, et l'ordre entre elles est le cœur du problème. La première prépare, pour chaque succursale, une ligne par famille avec son chiffre de la semaine. La seconde agrège avec `ROLLUP (region, succursale)` et étale les familles en colonnes par agrégats conditionnels, une colonne par famille.

Les deux besoins se gênent pour une raison précise : le pivot met les familles en colonnes, donc il les fait disparaître du `GROUP BY`, alors que le `ROLLUP` a besoin d'une hiérarchie de colonnes regroupées pour produire ses niveaux. Si l'on tentait un `PIVOT` par-dessus un `ROLLUP`, la sous-requête source contiendrait les lignes de sous-total, que le pivot regrouperait à son tour, et les totaux se retrouveraient mélangés au détail dans les mêmes cases. L'ordre correct est donc l'inverse de l'intuition : on agrège avec le `ROLLUP` en gardant la famille comme axe de mesure, c'est-à-dire dans les agrégats conditionnels, et jamais comme colonne de

© 2026 iAlexMG. Tous droits réservés.

regroupement. Concrètement, `GROUP BY ROLLUP (region, succursale)` avec six colonnes `SUM(CASE WHEN famille = … THEN montant END)`. Les sous-totaux se calculent alors par famille automatiquement, puisque chaque colonne est un agrégat comme un autre.

L'étiquetage n'est pas cosmétique, et c'est le diagnostic E3 de ce cours en situation réelle. Sur la ligne de sous-total d'une région, la colonne succursale doit porter un libellé explicite du genre « Total Nord », posé par un `CASE WHEN GROUPING(succursale) = 1`, jamais un `NULL` ni un `COALESCE`. La raison est qu'un lecteur qui parcourt vingt-deux lignes ne doit pas avoir à déduire du gras ou de la position qu'une ligne est un total, et surtout qu'une succursale dont le nom manquerait dans le référentiel produirait autrement une ligne « Total » parasite au milieu du détail. Un rapport lu par des gestionnaires le lundi matin ne peut pas se permettre une ambiguïté qui se lit comme une erreur de chiffres.

Théorie : `Theorie_06_SQL.md`, sections 3, 5 et 7.

M2. Coopérative agricole Saint-Hyacinthe.

Aucune erreur n'a été levée parce qu'un `PIVOT` ne vérifie rien. Il ne compare pas la liste écrite en dur à ce qu'il trouve dans les données : il projette les valeurs demandées et ignore les autres, silencieusement. C'est le comportement documenté, pas un défaut du moteur. Pendant deux mois, le rapport a donc montré un total des cultures qui excluait le sarrasin, tout en présentant ce total comme le chiffre de la coopérative. Les trois membres concernés ont vu leurs livraisons disparaître d'un rapport auquel tout le monde se fiait, et le pire est que les colonnes affichées, elles, étaient parfaitement exactes.

La reconception répond aux trois questions.

Détecter une valeur nouvelle ne demande pas de deviner : on écrit un contrôle qui compare la liste des cultures présentes dans les données à la liste attendue, et qui échoue s'il en trouve une de plus. Ce contrôle tourne avant le rapport, pas après, et il échoue bruyamment. C'est exactement ce que fait un test d'acceptation de valeurs dans un outil de transformation, et c'est le seul mécanisme qui aurait transformé deux mois de silence en une alerte le premier jour.

Faut-il que le rapport s'adapte tout seul ? Ma réponse est non, et c'est contre-intuitif. Un rapport financier dont les colonnes apparaissent sans que personne ne l'ait décidé est un rapport dont on ne peut plus comparer deux éditions. Ce qu'on veut, ce n'est pas l'adaptation automatique, c'est la détection automatique suivie d'une décision humaine. La différence est celle entre un système qui vous cache un changement et un système qui vous oblige à le regarder.

La responsabilité du croisement, enfin, doit sortir de la requête. La couche de données livre le format long, une ligne par membre, culture et période. Le croisement se fait dans l'outil qui affiche, qui découvre les colonnes à chaque rafraîchissement et n'a donc rien à figer.

Au gestionnaire qui demande d'ajouter le sarrasin, je réponds oui, et je le fais dans l'heure, parce que le besoin immédiat est légitime et urgent. Puis je lui explique que ce correctif règle le cas du sarrasin et aucun autre, que la même chose se reproduira à la prochaine culture, et que ce qu'il faut corriger, c'est le mécanisme qui a permis à deux mois de chiffres faux de passer inaperçus. Présenter le contrôle et le format long comme la vraie demande, pas comme une amélioration technique, est la partie du travail qui décide si elle se fera.

Théorie : `Theorie_06_SQL.md`, section 7.

M3. `(avancé)` Groupe hôtelier Charlevoix.

La démarche se conduit par grains décroissants, et l'ordre n'est pas négociable : comparer des détails avant d'avoir compris l'écart global fait perdre des journées.

© 2026 iAlexMG. Tous droits réservés.

On commence par vérifier que les deux totaux portent bien sur la même chose. Même période, définie de la même façon aux bornes, même périmètre d'établissements, même définition du chiffre, taxes et frais compris ou non. Beaucoup d'écarts de trois pour cent s'expliquent ici, et il serait absurde de descendre au détail avant de l'avoir écarté.

On descend ensuite d'un cran, à l'établissement. Un écart réparti uniformément sur les neuf établissements ne dit pas la même chose qu'un écart concentré sur un seul : le premier oriente vers une différence de définition ou de règle, le second vers un problème de données localisé. Cette seule bifurcation élimine la moitié des hypothèses.

On descend enfin à la transaction, mais seulement sur le périmètre que l'étape précédente a désigné, et par une comparaison ensembliste : les identifiants présents d'un côté et pas de l'autre, dans les deux sens. C'est la seule façon de distinguer « il manque des lignes » de « les lignes sont les mêmes mais les montants diffèrent », qui appellent des enquêtes complètement différentes.

Les quatre causes classiques, dans l'ordre où je les testerais.

Le périmètre et les filtres. Les réservations annulées, les nuitées offertes, les séjours à cheval sur deux mois : chaque chaîne a fait un choix, et les deux choix sont rarement identiques.

La duplication par jointure. Une jointure vers une table à plusieurs lignes par clé multiplie les montants sans rien signaler. C'est la cause la plus fréquente d'un écart qui va toujours dans le même sens, et elle se détecte en comptant les lignes avant et après chaque jointure.

Le décalage temporel. L'entrepôt est alimenté la nuit, la base de réservation vit en continu, et les corrections tardives arrivent après le chargement. Deux chiffres justes peuvent différer simplement parce qu'ils sont datés différemment, ce qui se vérifie en rejouant la comparaison sur une période close depuis longtemps.

Enfin, la cause qui tient à ce cours : la ré-agrégation d'un résultat qui contient déjà des sous-totaux. Si le rapport financier est bâti sur un `ROLLUP` et que quelqu'un en aval somme la colonne sans filtrer les lignes de synthèse, le total gonfle exactement de la façon décrite au diagnostic E1. Un écart de cette nature a une signature reconnaissable : il est un multiple presque exact, et non un pourcentage quelconque. Trois pour cent n'y ressemble pas, ce qui la rend peu probable ici, mais elle se teste en une requête et mérite d'être écartée tôt.

Pour éviter la récidive, sachant qu'on ne peut supprimer ni l'un ni l'autre des deux rapports, la recommandation tient en trois points. Les deux chaînes doivent partir de la même définition écrite du chiffre d'affaires, et cette définition doit vivre dans un endroit unique et versionné, pas dans la tête de deux équipes. Un contrôle de réconciliation automatique doit tourner chaque mois et comparer les deux totaux, avec un seuil d'alerte : la valeur d'un tel contrôle n'est pas d'empêcher l'écart, c'est de le signaler le jour où il naît plutôt que le jour où deux services se contredisent en réunion. Et le rapport des opérations devrait, à terme, se brancher sur l'entrepôt plutôt que sur la base de réservation, non pas pour supprimer un rapport mais pour supprimer une source de vérité concurrente. C'est là le vrai problème : l'écart n'est qu'un symptôme du fait que l'entreprise a deux chiffres officiels.

Théorie : `Theorie_06_SQL.md`, sections 5 et 9.

© 2026 iAlexMG. Tous droits réservés.